

Proof-of-Contribution System Progress Report

Project: Smart Contract Supported Proof-of-Contribution and Soulbound Achievement Badge System

Student: Abdullah Gokalp Ciray

Base Document: DSRM Proposal, "Blockchain-Based Verification of Contribution Ownership in Group Projects"

Current Version: Local MVP with smart contract, CLI, in-toto attestation report, local blockchain, and local explorer

Date: May 7, 2026

1. Executive Summary

The project started from the DSRM proposal titled Smart Contract Supported Proof-of-Contribution and Soulbound Achievement Badge System. The original motivation was to create a fairer and externally verifiable way to record individual contributions in university group projects, hackathons, and open-source teams.

The current implementation has reached a working local MVP. The system now scans a Git commit range, generates a contribution report in a standard attestation format, hashes that report, records the hash on a smart contract, collects threshold-based approvals from authorized approvers, finalizes the report, and verifies that the local report file still matches the on-chain hash.

The most important design decision from the proposal has been preserved: the actual evidence file is not stored on-chain. Instead, the chain stores immutable evidence traces such as report hashes, commit hashes, and URIs.

2. Original Proposal Baseline

The proposal identified the following core problem:

Quote: In group projects, it is difficult to later verify the real contribution of each team member. Existing task tools may track tasks, but they do not provide immutable, timestamped, externally verifiable records of approved contribution evidence.

The proposed solution was a blockchain-based Proof-of-Contribution platform where:

- Contribution evidence is submitted by project members.
- Authorized users approve the evidence.
- Approved records are written to blockchain as immutable records.
- Contribution records are linked to wallet addresses.
- Evidence is represented by GitHub links, file hashes, report hashes, or IPFS CIDs.
- Contribution profiles and non-transferable Soulbound badges are eventually issued.

The proposal also used the DSRM approach and defined evaluation goals such as verifiability, immutability, authorization, duplicate prevention, and non-transferability of badges.

3. Current Implemented Scope

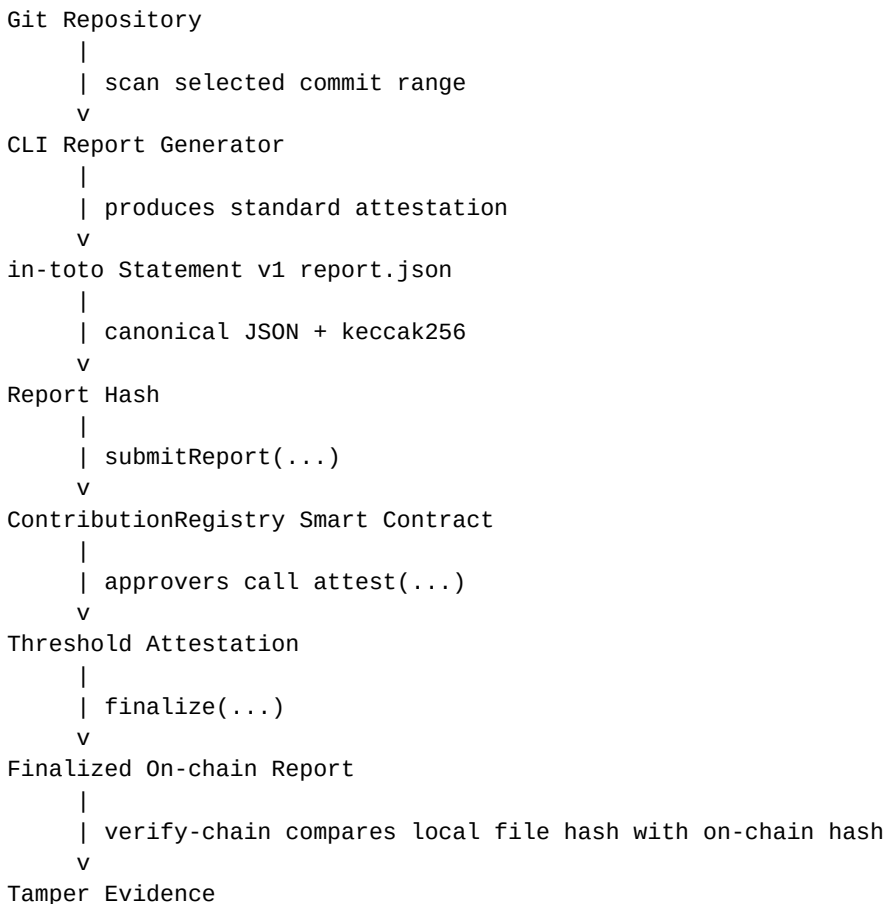
The current version implements the core Proof-of-Contribution verification flow at report level.

Implemented components:

- Solidity smart contract: ContributionRegistry
- TypeScript CLI for report generation and blockchain operations
- Git-based contribution scanning
- in-toto Statement v1 report format
- Deterministic report hashing
- On-chain report hash submission
- Threshold-based approver attestation
- Report finalization
- Chain-to-file hash verification
- Local Hardhat blockchain
- Lightweight local explorer
- Smart contract tests

The system is currently local-first. It does not depend on Polygon Amoy or any public testnet for demonstration. A public deployment can be done later once the local behavior is finalized.

4. Architecture of the Current System



5. Smart Contract Layer

The smart contract is located at:

contracts/contracts/ContributionRegistry.sol

The smart contract is used as the trust layer of the system. It does not calculate contribution quality. Instead, it records approved evidence hashes and controls the approval/finalization process.

Implemented smart contract functions:

- createRepo: creates a repository record with owner, approver list, and threshold.
- submitReport: stores report metadata and the report hash on-chain.
- attest: allows authorized approvers to approve a pending report.
- finalize: marks the report as finalized once the attestation threshold is met.
- reports: exposes stored report state for verification and explorer display.

On-chain data currently includes:

- repold
- contributor
- commitSha
- reportHash
- uri
- policyId
- attestationCount
- status
- submittedAt

6. Report Format and Standardization

The earlier custom JSON report format was replaced with in-toto Statement v1, which is a standard attestation envelope used in software supply-chain security.

Current report structure:

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [
    {
      "name": "git commit range",
      "digest": {
        "gitCommitFrom": "...",
        "gitCommitTo": "..."
      }
    }
  ],
  "predicateType": "https://akadaltr.dev/attestation/contribution-report/v1",
  "predicate": {
    "project": "...",
    "range": "...",
    "policy": "...",
    "contributors": [
      {
        "identity": "...",
        "metrics": {
          "commits": 1,
          "filesChanged": 1,
          "additions": 1,
          "deletions": 0,
          "netLines": 1
        }
      }
    ]
  }
}
```

```

    }
  ]
}
}

```

This change directly addresses the feedback that the report should use a generally recognized data template instead of a fully custom structure.

7. CLI Layer

The CLI is located in:

```
cli/
```

The CLI performs all off-chain operations:

- Reads Git history.
- Generates report.json.
- Computes the report hash.
- Sends transactions to the smart contract.
- Runs attestation and finalization commands.
- Verifies local files against on-chain hash values.

Main CLI commands:

```

node dist/index.js scan --from <oldCommit> --to <newCommit> --out report.json
node dist/index.js hash --file report.json
node dist/index.js submit --commit <commitSha> --uri "local://report.json" --file report.json
node dist/index.js attest --report-id 1
node dist/index.js finalize --report-id 1
node dist/index.js verify-chain --report-id 1 --file report.json

```

8. Local Blockchain and Explorer

The project now includes a local development environment:

- Local blockchain: Hardhat local chain
- Local RPC: <http://127.0.0.1:8545>
- Local explorer: <http://127.0.0.1:8787>

The local explorer shows:

- Chain ID
- Latest blocks
- Transactions
- Repo creation events
- Report submission events
- Attestation events
- Finalization events
- Report hash
- Attestation count
- Report status

This makes it possible to demonstrate the system without relying on Amoy testnet gas.

9. Current Demo State

The current local demo has successfully completed the following flow:

```
createRepo
-> scan Git commit range
-> generate in-toto report.json
-> hash report.json
-> submit report hash on-chain
-> two approvers attest
-> finalize report
-> verify local file hash against on-chain reportHash
```

Current finalized local report state:

```
reportId: 1
status: 1
status meaning: Finalized
attestationCount: 2
reportHash: 0xeedc69326f002aec873e18b18ae4e764161e10d17fce03b1fc9301ab400451d1
uri: local://report.json
```

The verify-chain command currently returns:

```
OK - file hash matches on-chain reportHash
```

This means that the local report.json file still matches the hash stored in the smart contract.

10. Technologies Used

The current implementation uses:

- Solidity for the smart contract
- Hardhat for local blockchain, compilation, and tests
- TypeScript and Node.js for the CLI
- ethers.js for blockchain interaction
- commander for CLI command definitions
- in-toto Statement v1 as the report attestation format
- JSON Schema for report structure validation
- Local HTTP explorer for visualizing local chain state

11. Evaluation Against Proposal Criteria

Proposal Criterion	Current Status	Notes
Verifiability	Implemented	Report hash, commit hash, URI, status, and approvals can be verified.
Immutability	Implemented for report hash	Once submitted, the on-chain report hash cannot be silently changed.
Authorization	Partially implemented	Approver-only attestation is implemented. More granular project/task roles remain future work.
Duplicate prevention	Implemented at report-hash level	The same report hash cannot be submitted twice for the same repo.

Evidence hash / CID model	Implemented	The system stores reportHash and uri; future deployment can use IPFS URI.
Contribution profile	Partially implemented	Report contains contributor metrics, but profile UI is not yet a full feature.
Soulbound badges	Not yet implemented	Still future work from the original proposal.
Web interface	Replaced by local explorer for MVP	The current explorer demonstrates chain state, but it is not a full user-facing app yet.

12. Differences From the Original Proposal

The original proposal described a broader project/task/contribution/badge system. The current MVP focuses on the core verification primitive first:

```
contribution report
-> attestation document
-> hash
-> on-chain approval
-> finalization
-> verification
```

This is a narrower but stronger foundation. Instead of building a full UI and badge system immediately, the implementation first proves that tamper-resistant contribution evidence can be generated, stored, approved, finalized, and verified.

Main pending items from the proposal:

- Task-level contribution submission
- Project member management
- Full web interface
- Soulbound badge minting
- Badge transfer restriction tests
- IPFS or Arweave storage integration
- Public network deployment

13. Next Steps

Recommended next development steps:

1. Add task-level data structures to the smart contract.
2. Add member management and role-based authorization.
3. Add CLI commands for approver management.
4. Add IPFS storage for report.json.
5. Extend the explorer into a web interface.
6. Implement Soulbound badge minting.
7. Add tests for badge non-transferability.
8. Harden validation for CLI inputs and smart contract edge cases.
9. Deploy to a selected public testnet or production-compatible blockchain.

14. Conclusion

The project has progressed from a proposal into a working local MVP. It now demonstrates the most important blockchain-based proof-of-contribution concept: contribution evidence can be represented as a standard attestation document, hashed, stored on-chain, approved by authorized parties, finalized after a threshold is met, and later verified against the original file.

In its current form, the system is a functional blockchain-based contribution verification prototype. It preserves the main idea of the proposal while narrowing the implementation to the most important technical proof: tamper-evident, approved, and externally verifiable contribution records.